Great Tables: Typst Rendering Examples

```python
import numpy as np
import pandas as pd
from great_tables import GT, md, typst, loc, style

cohort_df = pd.DataFrame(
    {
        "Characteristic": [
            "N",
            "Age (mean)",
            "Documented Sex Female (%)",
            "Race/Ethnicity (%)",
            "\u00a0\u00a0\u00a0White",
            "\u00a0\u00a0\u00a0Black",
            "\u00a0\u00a0\u00a0Hispanic",
            "\u00a0\u00a0\u00a0Other",
            "Region (%)",
            "\u00a0\u00a0\u00a0Northeast",
            "\u00a0\u00a0\u00a0Midwest",
            "\u00a0\u00a0\u00a0West",
            "\u00a0\u00a0\u00a0South",
            "Stage at Diagnosis (%)",
            "\u00a0\u00a0\u00a0I/II",
            "\u00a0\u00a0\u00a0III",
            "\u00a0\u00a0\u00a0IV",
        ],
        "dev_cohort": [
            14760,
            72.1,
            45.4,
            np.nan,
            82.7,
            8.8,
            4.2,
            4.2,
            np.nan,
            20.3,
            13.6,
            34.2,
```

```
            31.9,
            np.nan,
            15.1,
            34.0,
            50.8,
        ],
        "val_cohort": [
            14620,
            71.9,
            46.8,
            np.nan,
            81.4,
            8.9,
            4.4,
            5.3,
            np.nan,
            20.2,
            13.2,
            34.5,
            32.2,
            np.nan,
            14.8,
            33.1,
            52.0,
        ],
    }
)

# Identify category header rows (have NaN values)
category_rows = cohort_df[cohort_df["dev_cohort"].isna()].index.tolist()

gt_cohort = (
    GT(cohort_df)
    .cols_label(
        dev_cohort=typst("*Development Cohort* \ 2010–2011 Diagnosis"),
        val_cohort=typst("*Validation Cohort* \ 2012–2013 Diagnosis"),
    )
    .fmt_number(columns=["dev_cohort", "val_cohort"], decimals=1)
    .fmt_integer(columns=["dev_cohort", "val_cohort"], rows=[0], use_seps=True)
    .sub_missing(columns=["dev_cohort", "val_cohort"], missing_text="")
    .cols_align(align="left", columns="Characteristic")
    .cols_align(align="right", columns=["dev_cohort", "val_cohort"])
    .cols_width(
```

```
        {"Characteristic": "200px", "dev_cohort": "160px", "val_cohort": "160px"}
    )
    .tab_style(
        style=style.text(weight="bold"),
        locations=loc.body(columns="Characteristic", rows=category_rows),
    )
    .tab_options(
        column_labels_font_weight="bold",
    )
)

gt_cohort
```

Table 1: Cohort summary statistics.

Characteristic	Development Cohort **2010–2011 Diagnosis**	Validation Cohort **2012–2013 Diagnosis**
N	14,760	14,620
Age (mean)	72.1	71.9
Documented Sex Female (%)	45.4	46.8
Race/Ethnicity (%)		
White	82.7	81.4
Black	8.8	8.9
Hispanic	4.2	4.4
Other	4.2	5.3
Region (%)		
Northeast	20.3	20.2
Midwest	13.6	13.2
West	34.2	34.5
South	31.9	32.2
Stage at Diagnosis (%)		
I/II	15.1	14.8
III	34.0	33.1
IV	50.8	52.0

```
perf_df = pd.DataFrame({
    "Method": [
        "Baseline",
        r'Group $alpha = 0.1$ \ Pop $alpha = 0.1, 0.2$',
        r'Max $alpha = 0.1$',
        r'Group $alpha = 0.2, 0.3$',
        r'Max $alpha = 0.2, 0.3, 0.4, 0.5, 0.6$',
       r'Group $alpha = 0.4, 0.5, 0.6, 0.7,$ \ #context h(measure([Group $alpha
=$]).width + 0.3em) $0.8, 0.9, 0.99$ \ Pop $alpha = 0.3, 0.4, 0.5, 0.6,$ \
#context h(measure([Pop $alpha =$]).width + 0.3em) $0.7, 0.8, 0.9, 0.99$',
        r'Max $alpha = 0.7, 0.8, 0.9, 0.99$',
        r'ExpGrad, Ep $= 0.1$',
        r'ExpGrad, Ep $= 0.01$',
        r'Grid, CW $= 0.25$',
        r'Grid, CW $= 0.5$',
        r'Grid, CW $= 0.75$',
    ],
     "Accuracy": [0.95, 0.95, 0.95, 0.93, 0.94, 0.92, 0.91, 0.96, 0.93, 0.32,
0.11, 0.11],
    "Sensitivity": [0.36, 0.37, 0.29, 0.38, 0.31, 0.39, 0.33, 0.15, 0.13, 0.90,
0.95, 0.95],
      "Mean": [0.09, 0.02, 0.08, 0.03, 0.08, 0.02, 0.08, 0.04, 0.02, 0.01,
0.01, 0.01],
      "Black": [0.06, 0.00, 0.06, 0.00, 0.07, 0.00, 0.08, 0.03, 0.01, 0.00,
0.02, 0.02],
      "Asian": [0.24, 0.19, 0.17, 0.23, 0.11, 0.17, 0.00, 0.12, 0.10, 0.12,
0.00, 0.00],
     "HNW": [0.07, 0.00, 0.07, 0.00, 0.08, 0.00, 0.13, 0.02, 0.00, 0.00, 0.00,
0.00],
})

(
    GT(perf_df)
    .fmt_typst(columns="Method")
     .fmt_number(columns=["Accuracy", "Sensitivity", "Mean", "Black", "Asian",
"HNW"], decimals=2)
    .tab_spanner(label="Unfairness", columns=["Mean", "Black", "Asian", "HNW"])
    .cols_align(align="left", columns="Method")
      .cols_align(align="center", columns=["Accuracy", "Sensitivity", "Mean",
"Black", "Asian", "HNW"])
    .cols_width({"Method": "200px"})
    .tab_source_note(
        typst(
```

```
            "*Note:* Novel fair penalized regressions were defined based on the "
               "multi-group unfairness synthesis mechanism: population-weighted
average "
              "(Pop), group-weighted average (Group), and maximum (Max) as well
as "
            "fairness score weight ($alpha$). Fair reductions comparisons were "
               "exponentiated gradient (ExpGrad) or grid search (Grid) methods,
varying "
          "constraint flexibility (Ep) or constraint weight (CW), respectively. "
             "'HNW' is Hispanic non-white."
        )
    )
    .tab_options(column_labels_font_weight="bold")
)
```

Table 2: **Performance for Baseline Unpenalized Regression, Novel Penalized Regressions, and Fair Reductions Methods.**

			Unfairness			
Method	**Accuracy**	**Sensitivity**	**Mean**	**Black**	**Asian**	**HNW**
Baseline	0.95	0.36	0.09	0.06	0.24	0.07
Group $\alpha = 0.1$ Pop $\alpha = 0.1, 0.2$	0.95	0.37	0.02	0.00	0.19	0.00
Max $\alpha = 0.1$	0.95	0.29	0.08	0.06	0.17	0.07
Group $\alpha = 0.2, 0.3$	0.93	0.38	0.03	0.00	0.23	0.00
Max $\alpha = 0.2, 0.3, 0.4, 0.5, 0.6$	0.94	0.31	0.08	0.07	0.11	0.08
Group $\alpha = 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 0.99$ Pop $\alpha = 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 0.99$	0.92	0.39	0.02	0.00	0.17	0.00
Max $\alpha = 0.7, 0.8, 0.9, 0.99$	0.91	0.33	0.08	0.08	0.00	0.13
ExpGrad, Ep $= 0.1$	0.96	0.15	0.04	0.03	0.12	0.02
ExpGrad, Ep $= 0.01$	0.93	0.13	0.02	0.01	0.10	0.00
Grid, CW $= 0.25$	0.32	0.90	0.01	0.00	0.12	0.00
Grid, CW $= 0.5$	0.11	0.95	0.01	0.02	0.00	0.00
Grid, CW $= 0.75$	0.11	0.95	0.01	0.02	0.00	0.00

Note: Novel fair penalized regressions were defined based on the multi-group unfairness synthesis mechanism: population-weighted average (Pop), group-weighted average (Group), and maximum (Max) as well as fairness score weight (α). Fair reductions comparisons were exponentiated gradient (ExpGrad) or grid search (Grid) methods, varying constraint flexibility (Ep) or constraint weight (CW), respectively. 'HNW' is Hispanic non-white.

```
reg_df = pd.DataFrame({
    "Method": [
        "Average",
        "Covariance",
        r"Net Compensation#super[$dagger$]",
        r"Weighted Average#super[$dagger.double$]",
        r"Mean Residual \ Difference#super[$plus.circle$]",
        "OLS",
    ],
    "r2": [12.4, 12.4, 12.5, 12.6, 12.8, 12.9],
    "ratio_g": [0.996, 0.996, 0.980, 0.964, 0.895, 0.837],
```

```
    "ratio_gc": [1.001, 1.001, 1.006, 1.011, 1.032, 1.050],
    "comp_g": [-46, -46, -232, -411, -1208, -1872],
    "comp_gc": [4, 4, 34, 62, 188, 293],
    "mrd": [-50, -50, -266, -473, -1396, -2165],
    "fair_cov": [6, 6, 31, 56, 164, 256],
})

(
    GT(reg_df)
    .fmt_typst(columns="Method")
    .fmt_percent(columns="r2", decimals=1, scale_values=False, rows=[0])
    .fmt_number(columns="r2", decimals=1, rows=[1, 2, 3, 4, 5])
    .fmt_number(columns=["ratio_g", "ratio_gc"], decimals=3)
    .fmt_currency(columns=["comp_g", "comp_gc", "mrd"], rows=[0], decimals=0)
    .fmt_integer(columns=["comp_g", "comp_gc", "mrd"], rows=[1, 2, 3, 4, 5])
    .fmt_integer(columns="fair_cov")
    .cols_label(
        Method="Method",
        r2=typst("$R^2$"),
        ratio_g=typst("$g$"),
        ratio_gc=typst("$g^c$"),
        comp_g=typst("$g$"),
        comp_gc=typst("$g^c$"),
        mrd=typst("Mean \ Residual \ Difference"),
        fair_cov=typst("Fair \ Covariance"),
    )
        .tab_spanner(label=typst("Predictive  \  Ratio"),  columns=["ratio_g",
"ratio_gc"])
         .tab_spanner(label=typst("Net  \  Compensation"),  columns=["comp_g",
"comp_gc"])
    .cols_align(align="center", columns=["r2", "ratio_g", "ratio_gc", "comp_g",
"comp_gc", "mrd", "fair_cov"])
    .cols_align(align="right", columns="Method")
    .tab_source_note(
        typst(
           r'$dagger lambda = 10000$, $dagger.double alpha = 0.2$, $plus.circle
lambda = 30000$ \ '
              r"_Note:_ Measures calculated based on cross-validated predicted
values and "
             r"sorted on net compensation. Best performing hyperparameters for
each "
               r"estimator (with respect to fairness measures) are displayed.
Performance "
```

```
                r"for covariance method was the same for all $m$. $g^c$ is the
complement of g."
        )
    )
    .tab_options(
        column_labels_font_weight="bold",
        table_body_hlines_style="none",
    )
)
```

Table 3: Performance of Constrained and Penalized Regression Methods

Method	R^2	Predictive Ratio		Net Compensation		Mean Residual Difference	Fair Covariance
		g	g^c	g	g^c		
Average	12.4%	0.996	1.001	-$46	$4	-$50	6
Covariance	12.4	0.996	1.001	–46	4	–50	6
Net Compensation†	12.5	0.980	1.006	–232	34	–266	31
Weighted Average‡	12.6	0.964	1.011	–411	62	–473	56
Mean Residual Difference⊕	12.8	0.895	1.032	–1,208	188	–1,396	164
OLS	12.9	0.837	1.050	–1,872	293	–2,165	256

† $\lambda = 10000$, ‡ $\alpha = 0.2$, ⊕ $\lambda = 30000$

Note: Measures calculated based on cross-validated predicted values and sorted on net compensation. Best performing hyperparameters for each estimator (with respect to fairness measures) are displayed. Performance for covariance method was the same for all m. g^c is the complement of g.